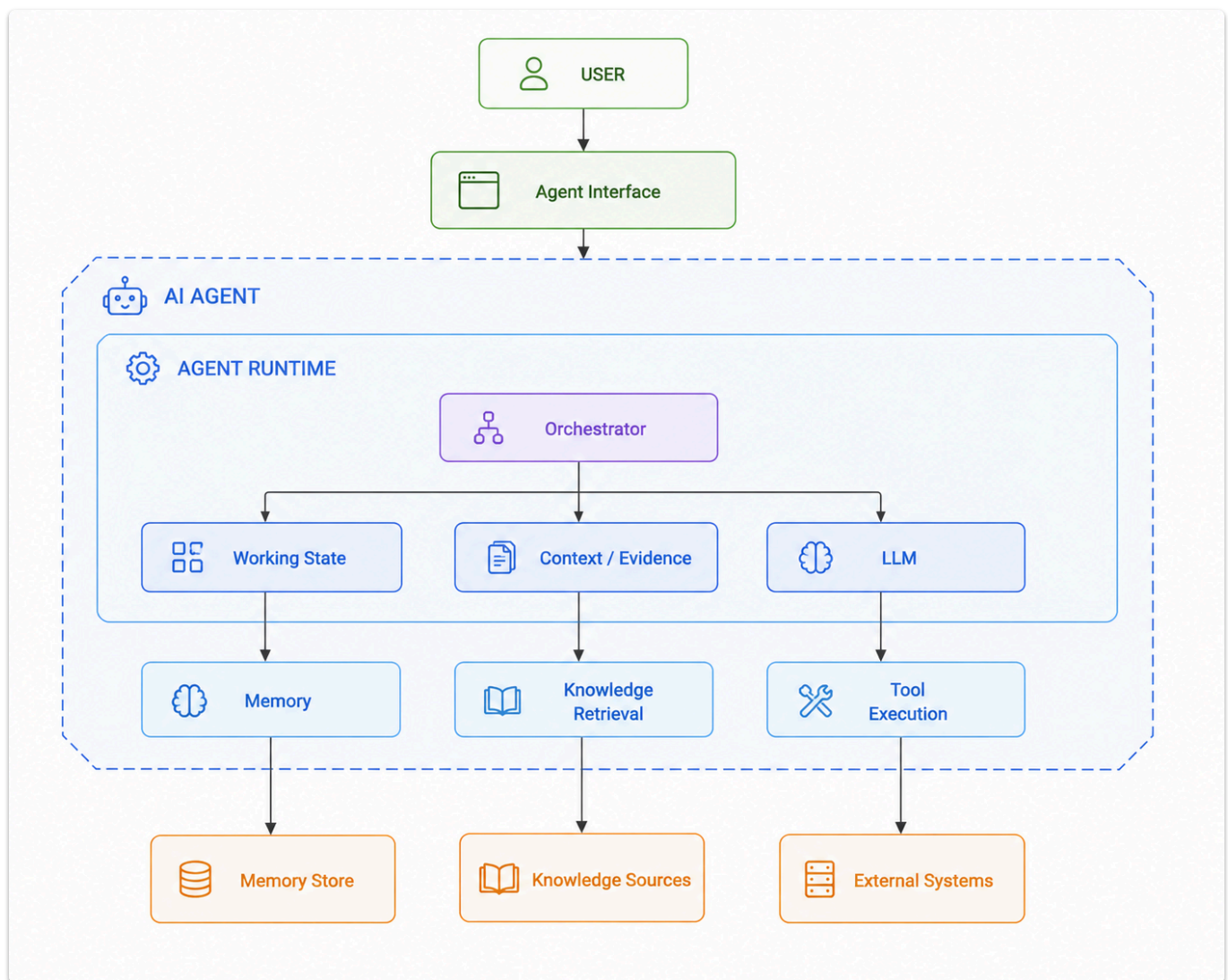


Task 02 - Báo cáo Research Memory AI

1. System Architecture



Agent Interface: tiếp nhận request từ User và trả response hoặc kết quả action về cho User

Agent Runtime: Là môi trường thực thi một task của Agent. Runtime chịu trách nhiệm điều phối execution, quản lý trạng thái hiện tại và chuẩn bị thông tin cần thiết cho reasoning
Runtime gồm:

- **Orchestrator** - điều phối execution.
- **Working State** - lưu trạng thái hiện tại của session/task/execution, bao gồm Working Memory.
- **Context / Evidence** - tổng hợp thông tin cần thiết cho reasoning.
- **LLM** - thực hiện reasoning và tạo decision/response.

Memory: Cung cấp khả năng lưu trữ và truy xuất các thông tin dài hạn có giá trị cho các task hoặc conversation trong tương lai

Knowledge Retrieval: Cung cấp khả năng truy xuất thông tin liên quan từ các knowledge sources được cung cấp

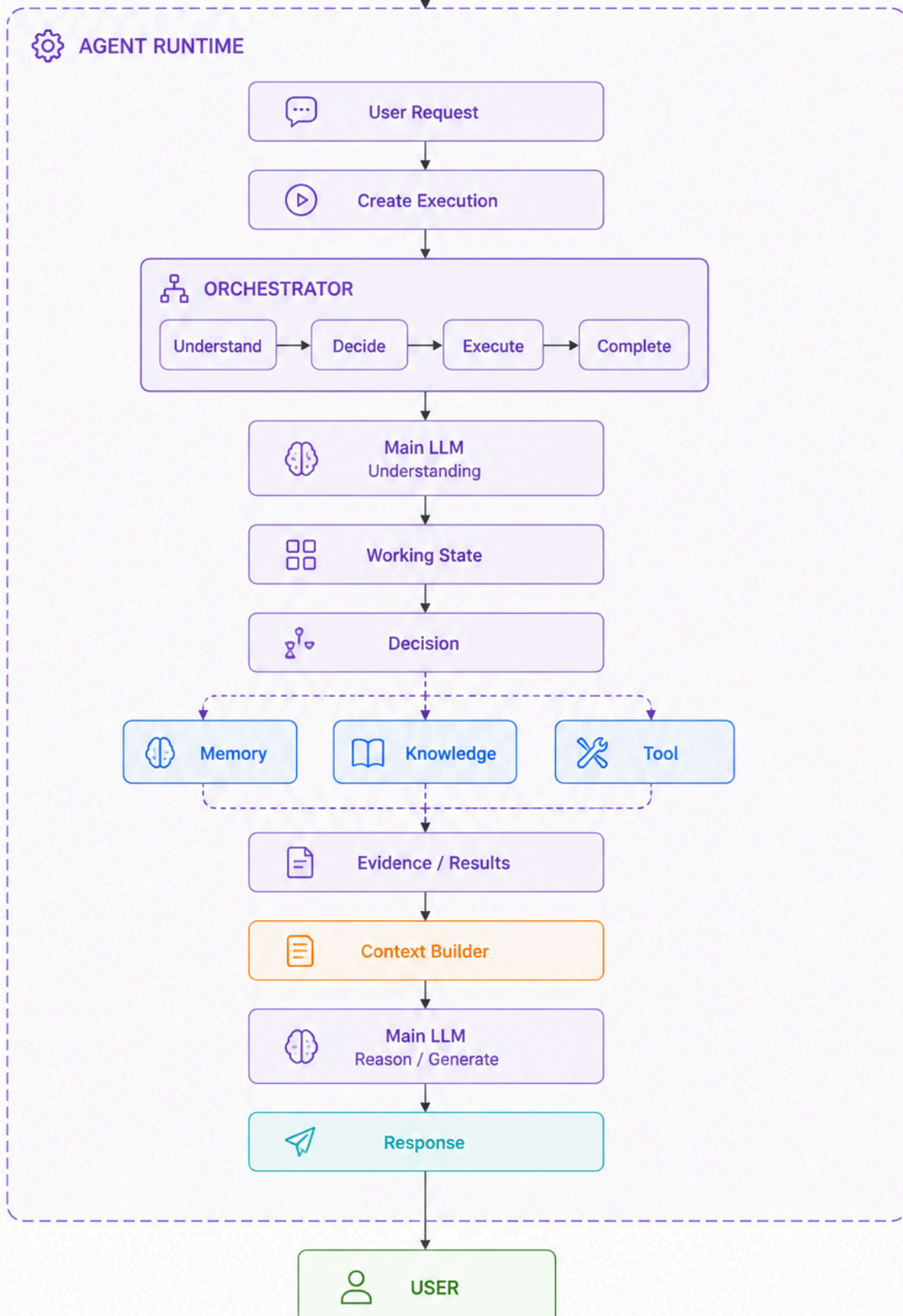
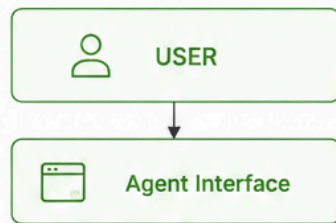
Tool Execution: Cung cấp khả năng sử dụng external capabilities để lấy dữ liệu hoặc thực hiện action

2. Agent Runtime Flow

Khi User gửi một request, Runtime sẽ phải làm những việc gì?

Note: Tạm thời ở đây chưa xử lý việc Agent Loop

Happy Path:

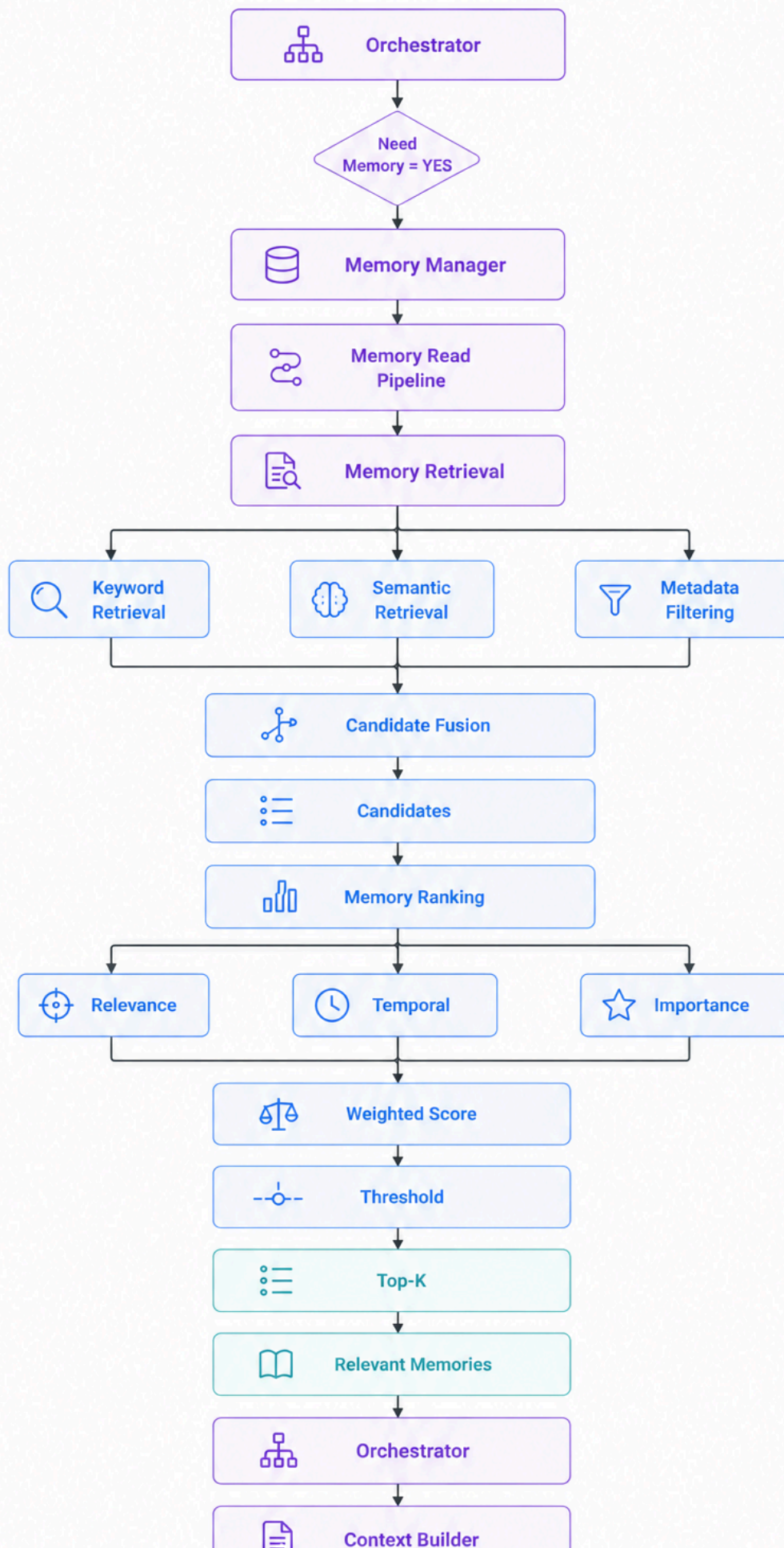


Failure Path: Chưa thiết kế

Note: Chốt Happy Path làm Main Flow để đơn giản trước.

3. Memory Manager

3.1. Read Pipeline



A - Memory Retrieval

Từ toàn bộ Memory Store, làm sao tìm ra một tập nhỏ memory có khả năng liên quan đến request hiện tại

Solution:

- Keyword
- VectorDB - Ưu: hiểu ngữ nghĩa - Nhược: similarity khác relevance, đôi khi lấy memory "gần nghĩa" nhưng không thực sự hữu ích
- Graph Retrieval - Có lợi khi memory có relationship phức tạp
- Hybrid Retrieval

Selected solution:

- **Stage 1:** Hybrid: Keyword + Metadata Filter + VectorDB
- **Stage 2:** Hybrid: Keyword + Metadata Filter + VectorDB + Graph Retrieval **nếu cần thiết**

Các bài toán cần giải quyết:

- **Problem 1 - Retrieval Scope & Metadata Filtering:** Retrieval được phép tìm trong phạm vi nào và cần thu hẹp search space trước khi retrieval.
-> **Solution:** Thiết kế data memory có *user_id, memory_status, time_range,...* User A thì không gọi memory của User B.
- **Problem 2 - Candidate Fusion:** Hai retrieval trả về 2 tập kết quả thì cần xử lý duplicate, retrieval score, source, candidate limit
-> **Solution:** Thêm *memory_id*. Chuẩn hoá score, sau đó thì tính điểm và lọc lại.
- **Problem 3 - Cold Start:** User mới chưa có memory thì retrieval sẽ giải quyết như nào?
-> **Solution:** Return Empty, sau đó Runtime tiếp tục xử lý request mà không có long-term memory. Lúc này có thể dựa vào **Working State**.
- **Problem 4 - Empty/Low-quality Retrieval:** Có candidate nhưng tất cả đều rất không liên quan
-> **Solution:** Thiết đặt một threshold. $score < threshold \rightarrow discard$

B - Memory Ranking

Trong các candidate này, memory nào thực sự nên được ưu tiên sử dụng?

Solution:

- Weighted Score Ranking: Chấm điểm dựa trên *Relevance, Temporal, Importance, Memory Status* -> Weighted Score -> Sort -> Top - K

- LLM-based Reranking
- Cross-Encoder

Selected solution:

- **Stage 1:** Weighted Score Ranking + Hard Constraints + Threshold + Top-K
Với Scoring signals: Relevance, Temporal, Importance
Hard constraints: Superseded, Deleted, Inactive -> discard
- **Stage 2:** Cross-Encoder **nếu cần thiết**

Các bài toán cần giải quyết:

- **Problem 1 - Relevance Scoring:** Candidate này liên quan đến query hiện tại đến mức nào?
-> **Solution:** Điểm này lấy từ Memory Retrieval
- **Problem 2 - Temporal Relevance:** Memory mới có nên được ưu tiên hơn memory cũ không?
-> **Solution:** Time Decay: Memory càng cũ -> score giảm dần. Nhưng: không áp dụng decay giống nhau cho mọi loại memory
- **Problem 3 - Importance:** Memory nào có giá trị lâu dài hơn
-> **Solution:** Vấn đề này sẽ được lưu khi Write Memory có thể dùng LLM để quyết định mức độ importance.
- **Problem 4 - Memory Status/Conflict:** Có memory nhưng nó thuộc: *deleted, deprecated, superseded, inactive* thì không được ranking
-> **Solution:** Sẽ filter theo status và sẽ có Lifecycle để cập nhật Memory nằm ở **Memory Write/ Lifecycle**
- **Problem 5 - Top-K Selection:** Nếu cắt theo K thì K bao nhiêu và nếu cắt K thì có thể bị mất memory cần thiết.
-> **Solution:** Có thể áp dụng Token Budget

C - Những bài toán khác

- **Problem 1 - Làm sao để biết query cần đọc những loại Memory nào?**
Note: Bài toán này sẽ được giải quyết sau

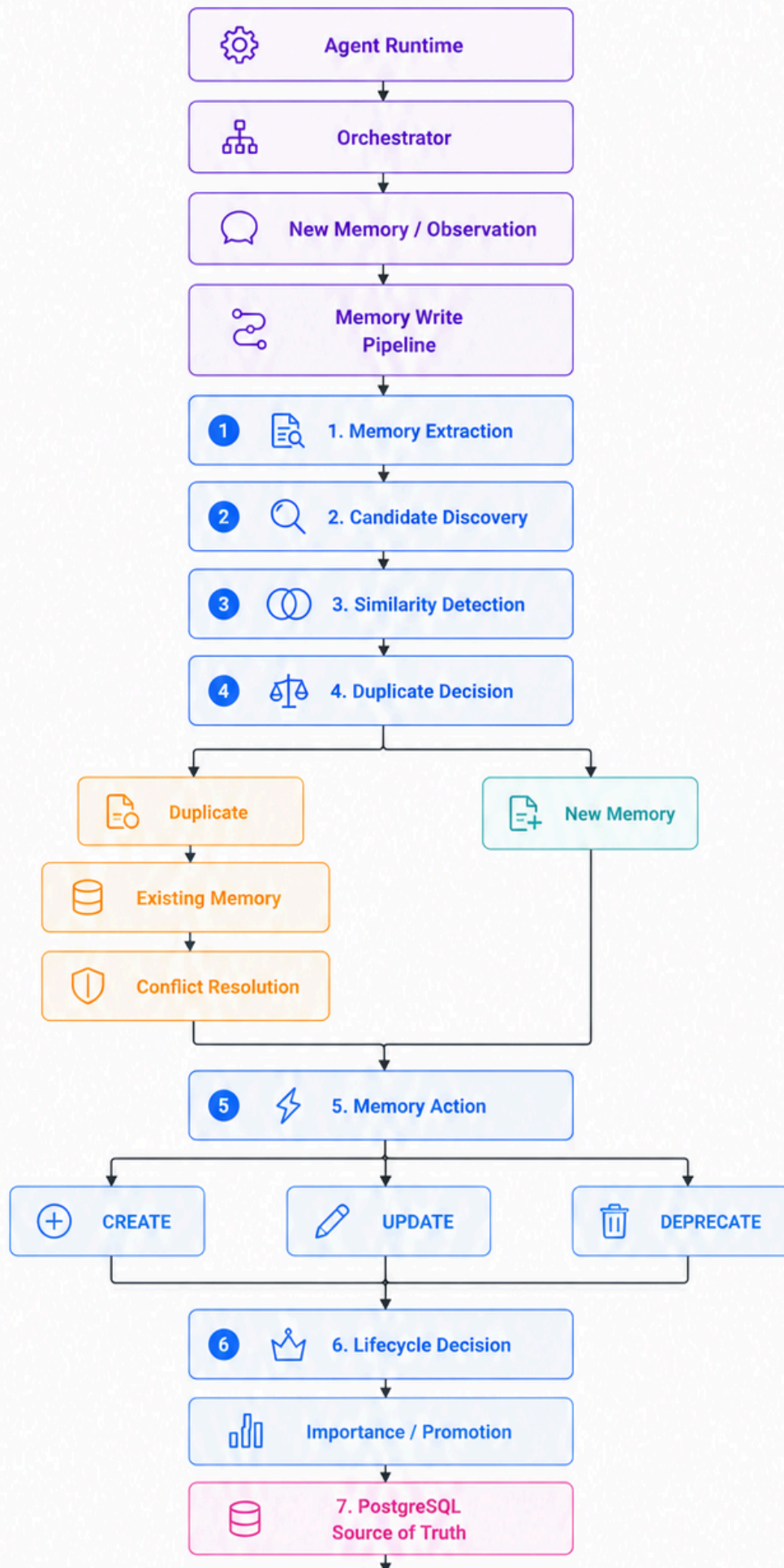
D - Technology

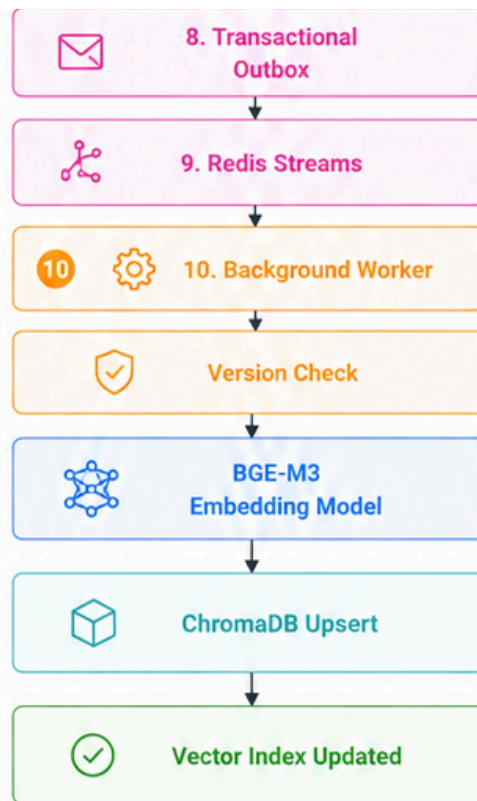
- Keyword Search - BM25
- Vector Search - BAAI/bge-m3 và ChromaDB

E - Evaluation

1. **Retrieval Quality:** Đánh giá Memory Retrieval có tìm được memory cần thiết hay không
Recall@K, Precision@K, Hit Rate @K
Ban đầu thì ưu tiên Recall@K
2. **Ranking Quality:** Memory nào được đưa lên đầu. Trách nhiệm của Memory Ranking
Precision@K - Trong Top-K có bao nhiêu memory thực sự relevant
MRR - Memory đúng nằm ở vị trí nào
NDCG@K - Phù hợp khi memory có mức độ relevance khác nhau
3. **End-to-End Memory Utility:** Memory có giúp Agent hoàn thành task tốt hơn không
Đo task success rate giữa có và không có memory
4. **Performance**
Memory Retrieval là synchronous nên latency rất quan trọng.
P50 latency
P95 latency
P99 latency

3.2. Write Pipeline





A - Memory Extraction

Từ conversation/interaction của user, thông tin nào đáng được chuyển thành Memory, và thông tin nào chỉ là dữ liệu tạm thời nên bỏ qua?

Solution:

1. **Rule-based:** Dùng các pattern/rule để phát hiện
2. **LLM-based Extraction:** Ưu: hiểu ngữ nghĩa, xử lý câu tự nhiên tốt,, có thể phát hiện implicit information, dễ mở rộng memory type - Nhược: latency, cost, có thể hallucinate, cần validation.
3. **Hybrid Rule + LLM:** Rule xử lý những trường hợp rõ ràng, LLM xử lý phần semantic
4. **Model Classifier**

Selected Solution:

- LLM-based Memory + Structured Output

B - Memory Validation

Các bài toán chính:

- Memory có đủ thông tin để lưu không?
- Output có đúng cấu trúc không?

- Có phải thông tin rõ ràng, có ý nghĩa không?
- Có phải thông tin vô nghĩa hoặc chỉ mang tính tạm thời không?
- Có đủ chắc chắn để lưu không?

Selected Solution: Schema Validation + basic Rule Validation -> Valid/Invalid

C - Memory Deduplication

Bài toán chính: Sau khi Extraction + Validation, ta có Memory mới, nhưng trong DB đã tồn tại. Hai memory này khác câu chữ nhưng cùng 1 information

Chia thành các vấn đề nhỏ:

- Tìm những memory cũ có khả năng trùng
- Xác định New Memory có thực sự trùng không?
- Duplicate hay không?
- Skip/Merge/Continue

Deduplication không nên scan toàn bộ Memory của user - Ta phải dùng retrieval để tìm candidate duplicates

Selected Solution:

- **Stage 1:** Tận dụng Vector search ở Read Pipeline
- **Stage 2:** Hybrid thêm LLM vào **nếu cần thiết**

D - Conflict Resolution

Bài toán: Memory mới và Memory cũ sẽ được xử lý như thế nào.

Có 5 action chính: SKIP, UPDATE, MERGE, DEPRECATE + CREATE, DELETE

- **SKIP:** khi không có information mới
- **UPDATE:** Memory mới chứa phiên bản mới hơn/ chính xác hơn của cùng một fact
Vấn đề: Làm sao để biết New thực sự là phiên bản mới hơn - *timestamp, source, confidence và semantic comparison* sẽ giải quyết
- **MERGE:** Hai memory đều có information hữu ích
- **DEPRECATE + CREATE:** Đây là pattern rất quan trọng khi fact thay đổi theo thời gian

Ví dụ:

Old: User đang học Python

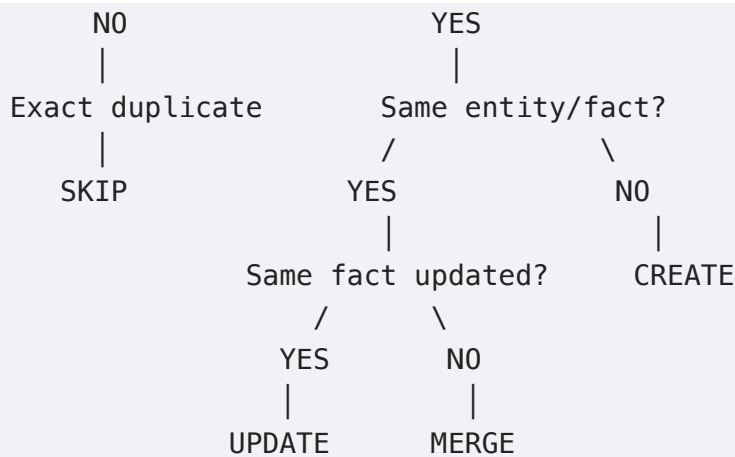
New: User chuyển sang học Rust

-> Không nên **DELETE old** vì lịch sử có thể vẫn có giá trị

- **DELETE:** Dùng khi memory thực sự không còn giá trị hoặc cần loại bỏ

New information?

/ \



Solution:

1. Rule-based

Dựa vào: *timestamp, source, confidence, memory type, status* để xử lý

2. LLM-based

3. Versioning/Temporal Resolution

Thay vì cố xoá conflict, lưu lịch sử thay đổi

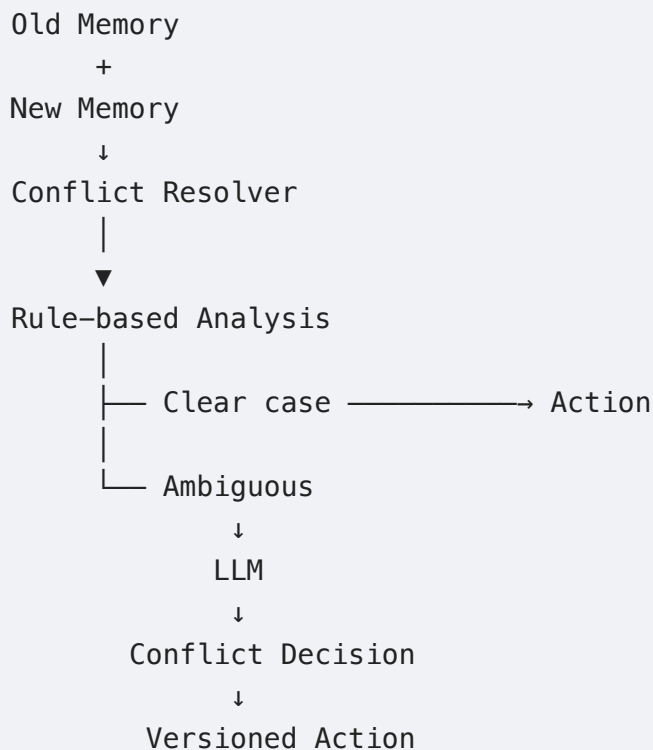
Retrieval mặc định lấy status = active nhưng history vẫn tồn tại

Ưu: giữ được lịch sử, giải quyết temporal conflict rất tốt, không mất information

Nhược: data model phức tạp

4. **Hybrid Rule + LLM + Versioning

Selected Solution: Hybrid



Ví dụ:

Old Memory:

User thích câu trả lời dài.

New Memory:

User muốn câu trả lời ngắn gọn.

Step 1 - Detect same subject

Hai memory đều nói về: response preference -> có khả năng conflict

Step 2 - Check temporal information

Old timestamp = Jan 1

New timestamp = Aug 10

-> New mới hơn

Step 3 - Check confidence/source

Old:

confidence = 0.7

New:

confidence = 0.95

-> New đáng tin hơn

Step 4 - Resolution

Old → deprecated

New → active

Vấn đề: Không phải conflict nào cũng giống nhau

- Temporal conflict: giá trị mới thay thế giá trị cũ
- Source conflict: hai nguồn đưa thông tin khác nhau
- Semantic Conflict: Hai memory thực sự mâu thuẫn
- Partial Conflict: Một phần giống, một phần khác

E - Memory Persistence

Lưu trạng thái memory cuối cùng một cách nhất quán để các lần Read sau có thể truy xuất được

Các bài toán chính:

- Lưu memory ở đâu
- Tạo/lưu embedding để Read có thể Vector Search
- metadata persistence: timestamp, importance, status,...
- Consistency: DB và VectorDB có bị lệch nhau không?
- Idempotency: Retry Write có tạo duplicate Memory không?

Problem 1 - Lưu memory ở đâu:

- **Stage 1:** Lưu 3 loại memory chung 1 DB nhưng có thêm memory_type và thêm một Vector DB
- **Stage 2:** Có thể tách 3 loại ra 3 DB nếu cần thiết - nếu 3 loại memory có lifecycle và schema hoàn toàn khác nhau

Problem 2 - Vector Indexing:

Có những vấn đề nhỏ bên trong:

- Dùng model nào để tạo vector - *bge-m3*
- Embed content nào? metadata xử lý ra sao? - *Solution: embed content*
- CREATE/ UPDATE/ MERGE thì vector thay đổi thế nào

Đối với UPDATE/MERGE:

Solution hiện tại là: embed mới và UPSERT vào ChromaDB để thay thế vector cũ của từng memory_id

Solution này tránh việc duplicate vector, PostgreSQL giữ lịch sử, Chromadb giữ trạng thái hiện tại, UPSERT giúp Idempotency

Có vấn đề là: Có trade-off là mất khả năng semantic search vào lịch sử và Eventual consistency

- Memory không còn active thì xử lý vector thế nào?
- PostgreSQL và ChromaDB đồng bộ thế nào?

Problem 3 - Consistency

Giả sử:

PostgreSQL
↓
ChromaDB

Ta làm synchronous:

1. UPDATE PostgreSQL
2. UPDATE ChromaDB

Có thể xảy ra:

```
PostgreSQL → SUCCESS  
ChromaDB    → FAILED
```

Kết quả:

```
PostgreSQL:  
M1 = "User thích câu trả lời ngắn"  
  
ChromaDB:  
M1 = vector của "User thích câu trả lời dài"
```

Solution: PostgreSQL Source of Truth + Async Indexing



Nhưng solution này gây ra 1 vấn đề là **Eventual Consistency** điều này có thể **chấp nhận được** với Memory Retrieval vì Memory Write không nhất thiết phải làm cho vector index cập nhật **ngay trong cùng millisecond**.

Ta ưu tiên:

```
Data correctness > immediate index freshness
```

PostgreSQL luôn là trạng thái chính xác.

Problem 4 - Idempotency của Vector Indexing

Solution:

```
1. Stable memory_id  
   ↓  
   chống duplicate record
```

2. Upsert



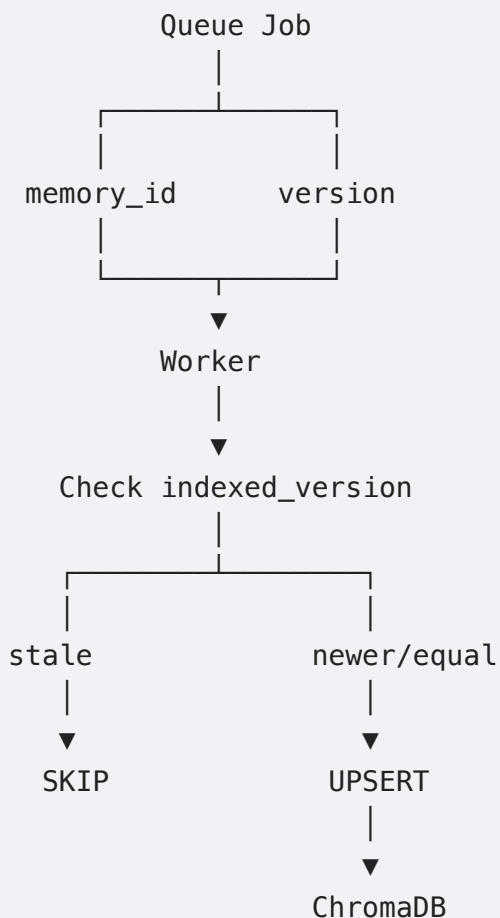
retry an toàn

3. Version



chống stale/out-of-order write

Flow:



Một solution cho stage trong tương lai: Transactional Outbox

Async Queue + Background Worker

Solution: Celery + Redis/RabbitMQ

Bài toán giải quyết: Nó ra đời để giải quyết những nhược điểm mà asyncio và BackgroundTasks chưa giải quyết được. Chuyển bài toán từ *quản lý coroutine trong một process sang quản lý job phân tán, bền vững và có khả năng mở rộng.*

- Bài toán không mất Task khi mà server sập.

- Bài toán tiếp theo là Retry. Nếu task đang làm bị lỗi thì sao có thử lại không hay là kết thúc luôn.
- Tách API khỏi Background Job. Với asyncio thì nếu Task càng nhiều thì API càng chậm. Còn đối với Celery thì FastAPI chỉ nhận request. Worker làm Job. API nhẹ hơn rất nhiều.
- Bài toán Scale. Nếu có 1000 Job thì asyncio sẽ có 1000 Task và FastAPI phải xử lý hết 1 mình. Còn Celery thì có thể chia thành nhiều Worker để xử lý
- Bài toán Monitoring. Với asyncio thì Task - không biết sống chết. Còn Celery thì ta có thể xem trạng thái của task
- Có thêm cơ chế Priority dành cho những task quan trọng hơn.

Cơ chế: Task được đẩy vào Message Queue (Redis/RabbitMQ). Celery Worker chạy trên process riêng biệt(hoặc server riêng) để xử lý task.

Khi nào thì dùng:

- Phải có kết quả mới trả Response: await
- Không cần kết quả. task nhẹ, mất cũng không sao: BackgroundTasks hoặc asyncio
- Task quan trọng, không được mất, cần retry, cần queue, cần nhiều worker: Celery + Redis.

Message Queue:

Bài toán giải quyết: Nếu có 10000 Task thì FastAPI phải xử lý từng task hết. API server ngoài việc nhận request còn phải lo quản lý hàng chục nghìn background task. API chỉ nên nhận request và trả về response. Lúc này **Message Queue** xuất hiện để lưu **Công việc** để Worker xử lý

Producer: Tạo job và đẩy vào Queue

Queue(Redis): Lưu job, chờ Worker lấy

Consumer(Celery): Lấy job và xử lý

Redis hoạt động như một Message Broker:

- Redis là một Database lưu dữ liệu trong RAM. Nó hỗ trợ rất nhiều kiểu dữ liệu: String, Hash, List, Set, Stream, ...

Queue chỉ là một cách sử dụng Redis.

Redis chỉ làm một việc là **Nhớ danh sách Job**

Redis có 2 cách phổ biến:

- Redis List: Lấy từ đầu. Thêm vào cuối FIFO
- Redis Streams: Có ID, ACK, Consumer Group -> mạnh hơn List rất nhiều

Note: Trước mắt hiểu được List sau đó thì học thêm Streams

Redis có một cơ chế là **Blocking Pop** tức là nếu Redis Queue rỗng thì Worker sẽ đợi chứ không hỏi job liên tục. Khi mà có job thì Redis sẽ đánh thức Worker giúp tiết kiệm tài nguyên.

Worker lấy Job như thế nào? ACK là gì?

ACK là cơ chế tôi xác nhận Job đã hoàn thành.

Luồng hoạt động: Queue -> Worker lấy job nhưng Redis chưa xóa -> Worker hoàn thành task , Worker gửi ACK. Redis -> OK -> Xóa Job. Nếu chưa hoàn thành thì task sẽ được giao cho Worker khác.

Nhưng có 1 vấn đề khác: Là nếu worker 1 đang gửi mà crash chưa kịp ACK mà Redis nghĩ chưa làm xong đưa cho worker 2 sau đó gửi email sau đó khách nhận 2 email.

Để giải quyết 1 khái niệm là Idempotency:

Ví dụ: Send Email. Worker nên kiểm tra *Email này đã gửi chưa*. Nếu rồi thì không gửi nữa.

Flow: Producer -> Redis Queue -> Worker lấy Job -> Đang xử lý -> Thành công -> ACK -> Xóa Job -> Nếu worker crash -> Không ACK -> Queue giao lại Job -> Worker mới chạy lại toàn bộ Job-> Retry giúp không mất Job nhưng đổi lại có thể phải làm lại từ đầu tốn thêm time.

Cách giải quyết vấn đề có thể phải làm lại từ đầu tốn thêm time:

- Cách 1: Chia nhỏ task ra để làm
- Cách 2: Checkpoint
- Cách 3: Thiết kế Idempotent

Một Job được tạo ra như thế nào?

Job chỉ chứa:

- ID
- Metadata
- Tham số nhỏ
- Không chứa dữ liệu lớn hay function

FastAPI -> Tạo Job Object -> Serialize (JSON/bytes) -> Redis Queue -> Worker lấy Job -> Deserialize -> Tìm Task theo tên -> Gọi function

Redis List & Blocking Pop

Redis dùng List để xây dựng Queue

Producer thêm Job bằng LPUSH hoặc RPUSH

Worker lấy Job bằng BLPOP để không phải xin job liên tục

BLPOP giúp Worker "ngủ" khi không có việc và được Redis đánh thức khi có Job

Redis có thể tự phân phối Job cho nhiều Worker đang chờ

Thiết kế Task

Một task phải:

- nhỏ
- độc lập

- có thể retry
- idempotent

Redis

Redis có nhiều Pattern trong đó có:

- Pub/Sub - không lưu message dùng để thông báo Realtime
- Stream - lưu message, có thể đọc sau dùng để event log/ history
- Persistency (RDB/AOF) - cơ chế ghi dữ liệu Redis vào phần cứng. RDB là chụp hình Redis lưu vào phần cứng. AOF thì là lưu liên tục.

Ưu: khắc phục được những nhược điểm của BackgroundTasks và asyncio

Nhược: Setup phức tạp hơn, Latency thêm 10-50ms (push vào queue)

Bài toán Concurrency

Nhiều Write/Worker cùng lúc đụng vào 1 memory

Concurrency

- 1. Versioning
 - Xác định đâu là version mới nhất
- 2. Locking
 - Có cần khóa Memory khi đang update?
- 3. Idempotency
 - Retry nhiều lần có gây sai không?
- 4. Race Condition
 - Hai operation cùng lúc thì kết quả thế nào?

Versioning:

Một Memory có thể bị update nhiều lần:

```
M1:v1
↓
M1:v2
↓
M1:v3
```

Nhưng các event có thể được xử lý **không đúng thứ tự**.

```
Worker A → v2  
Worker B → v3
```

```
v3 xử lý trước  
v2 xử lý sau
```

Nếu không có versioning:

```
v3 → ChromaDB  
v2 → ChromaDB
```

Các solution

- Timestamp-based versioning
 - Incremental integer version
 - Optimistic Concurrency Control
 - Database row version
- Chọn

Optimistic Versioning + integer **version**

Ví dụ:

```
M1  
version = 7
```

Job:

```
memory_id = M1  
version = 8
```

Worker chỉ apply nếu version hợp lệ.

Tại sao?

- đơn giản
- không cần lock trong phần lớn trường hợp
- xử lý tốt out-of-order event
- dễ kết hợp với Queue
- dễ debug

Locking:

Hai process cùng muốn update **cùng một Memory**:

```
Worker A → M1  
Worker B → M1
```

Có cần khóa M1 để chỉ một worker được xử lý?

Note: Bài toán này xử lý sau

Idempotency

Queue retry có thể khiến cùng một event chạy nhiều lần:

```
Job E123  
  
Attempt 1 → SUCCESS  
Worker crash  
Attempt 2 → SUCCESS
```

Nếu operation không idempotent:

```
M1  
M2
```

có thể xuất hiện duplicate.

Các solution

- Idempotency key
- Deduplication table
- Unique constraint
- Upsert
- Exactly-once processing

Chọn

At-least-once delivery + Idempotent Consumer

Cụ thể:

```
event_id = E123  
memory_id = M1
```

và:

```
PostgreSQL → unique event_id  
ChromaDB   → upsert(memory_id)
```

Race Condition:

Đây là hậu quả khi nhiều operation cùng tác động lên Memory.

Note: Bài toán này sẽ xử lý sau

3.3. Reflection & Memory Lifecycle

Các bài toán chính:

Reflection + Lifecycle

- 1. Episodic → Semantic
→ Khi nào một trải nghiệm trở thành kiến thức lâu dài?
- 2. Importance
→ Memory nào đáng giữ?
- 3. Decay
→ Memory nào giảm độ quan trọng theo thời gian?
- 4. Archive
→ Khi nào đưa Memory ra khỏi active set?
- 5. Memory Promotion / Demotion
→ Khi nào Memory trở thành long-term hoặc quay về trạng thái ít ưu tiên?

Problem 1 - Episodic → Semantic Memory

Không phải mọi thứ user nói đều đáng trở thành Long-term Memory.

Ví dụ:

```
User:  
"Hôm nay tôi đang ở quán Starbucks."
```

Đây có thể chỉ là:

Episodic / temporary

Trong khi:

"Tôi là developer và thường dùng Python."

Có thể trở thành:

Semantic / long-term

Ta cần quyết định:

Khi nào một observation nên được promote thành Memory lâu dài?

Solution

Có thể dùng:

1. **Rule-based**
 - explicit preference
 - identity
 - recurring behavior
2. **LLM-based classification**
 - LLM quyết định memory type / persistence
3. **Hybrid**
 - Rule filter trước
 - LLM quyết định các case khó

Chọn:

Hybrid

Vì không nên gọi LLM cho mọi observation.

```
graph TD
    A[Observation] --> B[Rule Filter]
    B --> C[Candidate?]
    C --> D[LLM Classification]
```



Semantic Memory

Problem 2 - Importance

Không phải Memory nào cũng quan trọng như nhau.

Importance ảnh hưởng đến:

- Ranking
- Context budget
- Retention
- Promotion

Solution chọn: Rule-based + optional LLM scoring

Problem 3 - Decay

Một Memory có thể **cũ dần**.

Solution

1. Time-based decay

Importance giảm theo thời gian.

2. Access-based decay

Memory được dùng thường xuyên thì giữ lâu hơn.

3. Reinforcement

Mỗi lần Memory được xác nhận lại → tăng strength.

4. Hybrid decay

Solution chọn: Hybrid đơn giản

```
importance
+
last_accessed
+
last_confirmed
+
created_at
```

Problem 4 - Archive

Memory không còn phù hợp để nằm trong **active retrieval set**, nhưng không nhất thiết phải xóa.

Solution

Có thể:

1. Hard Delete
2. Soft Delete
3. Archive
4. Cold Storage

Chọn

Soft Archive

State:

```
active
archived
deprecated
deleted
```

Retrieval:

```
status = active
```

Chỉ active Memory mới được retrieval mặc định.

Problem 5 - Promotion/Demotion

Một Memory có thể đi qua:

```
Candidate
  ↓
Active
  ↓
Reinforced
  ↓
Long-term
```

hoặc:

```
Active
  ↓
Stale
  ↓
Archived
```

Ví dụ:

```
User:
"Tôi thích câu trả lời ngắn."
```

Lần đầu:

```
candidate
```

User lặp lại 5 lần:

```
reinforced
```

→ promote:

```
long-term memory
```

Solution

Rule-based thresholds

Ví dụ:

```
confirmation_count >= 2
      ↓
promote
```

hoặc:

```
importance > threshold
      ↓
promote
```

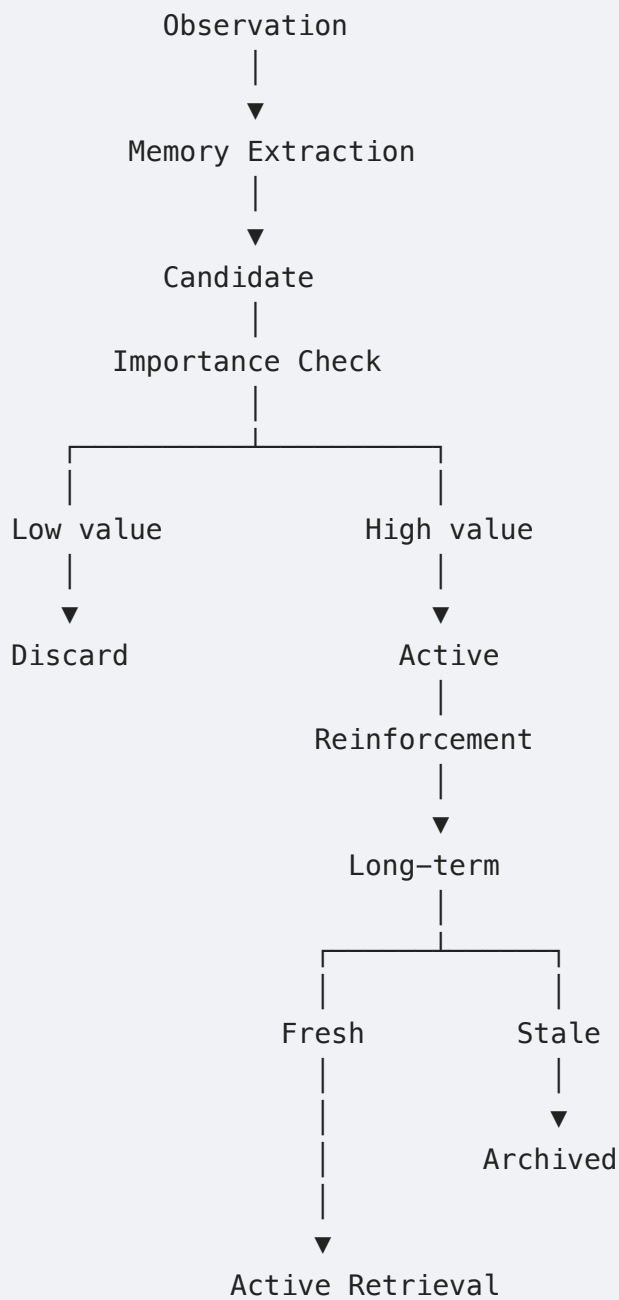
Chọn

Problem 6 - Reflection

Agent định kỳ nhìn lại các Memory để rút ra New Semantic Memory

Note: Bài toán này sẽ giải quyết sau

Flow tổng thể Lifecycle



3.4. Evaluation/Benchmark

Retrieval Evaluation

Recall@K là metric chính cho Retrieval - Trong K memory trả về, hệ thống có tìm được memory đúng không?

Ranking Evaluation

Memory nào được đưa lên đầu?

Metrics

Có thể dùng:

- Precision@K
- MRR
- NDCG@K

Chọn

| *NDCG@K + MRR*

Trong đó:

MRR tốt để đánh giá:

| *Memory đúng xuất hiện sớm đến mức nào?*

NDCG@K tốt hơn khi:

| *Có nhiều mức độ relevance khác nhau.*

Write Memory Evaluation

Extraction

Conversation

↓

Extracted Memory

Có extract đúng không?

Metrics:

Precision

Recall

F1

Deduplication

Ví dụ:

M1: User thích Python

New:

User rất thích Python

Hệ thống có nhận ra đây là cùng Memory không?

Ta đánh giá:

Duplicate Precision

Duplicate Recall

False Merge Rate

False Split Rate

Conflict Resolution

Ví dụ:

Old:

User thích câu trả lời dài.

New:

User thích câu trả lời ngắn.

Hệ thống có chọn đúng Memory mới không?

Metrics:

Conflict Resolution Accuracy

End-to-End Agent Evaluation

Ta tạo hai condition:

Agent + Memory

vs

Agent without Memory

Sau đó đo:

Task Success Rate

Ví dụ:

Without Memory → 68%

With Memory → 84%

→ Memory đang tạo ra giá trị thật.

Gọi là **Memory Utility**.

Thêm các metric đánh giá System Performance